

LEAF

A Layered Emission Assessment Framework for Cloud Deployments

Prakhar Singhal^{ID}, Shaunak Biswas^{ID}, Karthik Vaidhyanathan^{ID}, Akhila Matathammal^{ID}, and Ch Pavan Harshit^{ID}, International Institute of Information Technology (IIIT) Hyderabad

Ravi Kappagantu^{ID}, Srinivas Katakam^{ID}, and Jitendra Chougala^{ID}, Lloyds Technology Centre

// Existing sustainability assessment tools for carbon emissions produced by cloud infrastructure predominantly focus on postdeployment monitoring or require access to application code. To address this gap, we present a framework that enables early-stage estimation of carbon footprint from terraform configurations. //



AS CLOUD ADOPTION accelerates, so does its environmental footprint—prompting organizations to reconsider how infrastructure decisions

affect sustainability. With data center energy consumption projected to more than double by 2030,¹ the need for environmentally conscious

software development has become increasingly urgent.

Sustainability in software systems spans not just environmental concerns, but also economic, technical, and social dimensions, all of which influence architectural decisions and trade-offs.² While organizations are beginning to take these concerns seriously, tools like Cloud Carbon Footprint (CCF)^a are reactive, focusing on post-deployment monitoring. This rarely supports the proactive, sustainability-driven choices needed before provisioning. Consequently, developers and DevOps teams lack visibility into the environmental impact of their infrastructure decisions during predeployment.

The issue is particularly acute in environments where cloud provisioning is automated through infrastructure-as-code (IaC). Estimating carbon impact at the predeployment stage is difficult, a challenge analogous to predicting cost from resource topology.³ This is further complicated when application source code is unavailable,⁴ forcing reliance on coarse assumptions⁵ with limited accuracy or scalability. Recent work on sustainability smells in IaC⁶ identifies antipatterns but does not provide quantitative impact estimates.

This problem is particularly relevant for organizations like Lloyds Offshore Global Services Private Limited (hereafter referred to as Lloyds), which operate extensive cloud infrastructures using Terraform. Despite significant efforts in the direction guided by strong sustainability goals,⁷ Lloyds found it difficult to assess the environmental impact of architectural choices at the predeployment phase. Their need to estimate emissions from within existing IaC workflows without requiring

access to proprietary code reflected a broader industry demand.

To address this challenge, we collaborated with Lloyds to develop a layered emission assessment framework (LEAF): a lightweight framework that estimates carbon footprint directly from IaC configurations. LEAF models infrastructure behavior using a layered approach. It incorporates region-specific energy data and simulates workload scaling, all before any resources are provisioned. It enables sustainability-informed deci-

- *quantify the carbon impact* before provisioning infrastructure
- *compare configurations* (regions, services, instance types) from an environmental lens
- *embed sustainability checks* into existing Terraform workflows without relying on application source code.

This challenge is industry-wide: despite its extensive adoption, IaC lacks native support for sustainability analysis. This leads to decisions

like CCF, which derive emissions from cloud usage and billing data. By that stage, opportunities to optimize for sustainability are often limited or no longer feasible. Consequently, there is a need for proactive methods capable of estimating environmental impact directly from IaC specifications, such as Terraform files, before any resources are provisioned.

CH2: Insufficient Granularity in Predeployment Estimates

While existing predeployment traditional modeling-based tools like Carbonifer^b aim to provide resource-level granularity, their practical utility is diminished by incomplete support for many cloud services and resource types. This partial analysis means that unsupported resources are omitted from the calculation, leading to an unreliable aggregate forecast and preventing a comprehensive assessment of deployment tradeoffs. As a result, developers cannot confidently compare the environmental impact of different configurations.

There is a need for proactive methods capable of estimating environmental impact directly from IaC specifications, such as Terraform files, before any resources are provisioned.

sions during infrastructure planning, without disrupting existing workflows or requiring source-level access.

Challenges in IaC-Driven Sustainability

Cloud computing has transformed how enterprises build and manage infrastructure, with IaC tools like Terraform automating its provisioning. However, *environmental sustainability* often remains a secondary concern to performance, cost, and speed.

For organizations like Lloyds, which manages a vast and complex cloud footprint using IaC, this creates a critical gap. As a part of its strong commitment to sustainability, Lloyds' cloud engineering teams sought a solution to:

made without visibility into their environmental consequences.

When Lloyds approached us with the challenge of estimating environmental impact before deployment, we first reviewed the state of the art in sustainability-aware provisioning. From that analysis, we distilled five core challenges (CH):

CH1: Reactive versus Proactive Analysis

Most current solutions evaluate parameters, such as carbon emissions and energy consumption post-deployment. This reactive approach is common to both dashboards from major cloud providers (e.g., Google Cloud,⁸ Amazon Web Services (AWS)⁹) and open source estimators

CH3: Dependency on Code or Agents

Some methods require access to the application codebase or runtime agents. For example, tools like CodeCarbon^c instrument the running code itself to measure emissions. This creates a significant barrier for enterprises that treat code as proprietary or wish to analyze infrastructure independently.

CH4: The Need for Incremental Guidance in Sustainable IaC Development

Evaluating alternative IaC configurations, such as choosing between deployment in a green-energy region versus one geographically closer to

^b<https://github.com/carboniferio/carbonifer>

^c<https://github.com/mlco2/codecarbon>

target users, typically requires time-consuming analysis. While academic research has explored automated techniques, such as the use of genetic algorithms,¹⁰ these methods often work on the entirety of deployment. The Green Software Foundation’s (GSF) Software Carbon Intensity (SCI) specification¹¹ provides a standardized methodology supporting architectural decisions throughout the software life-cycle, and their Carbon Aware SDK⁴ demonstrates the value of time- and location-shifting workloads to reduce emissions. These approaches, although valuable in theory, fall short of addressing the more pragmatic needs of understanding the real-world trade-offs of specific configuration choices being actively worked on.

CH5: Disconnect Between Development Practices and Sustainability Data

Essential sustainability data, such as carbon-free energy percentages from Google Cloud¹² and real-time grid intensity from the GSF’s Carbon Aware SDK,⁴ is not natively integrated into IaC tools like Terraform. While efforts like the GSF’s Real-Time Cloud working group¹³ addresses standardization, this data remains external to the development workflow, rarely influencing provisioning decisions or automated checks.

Making IaC Carbon-Aware: The LEAF Approach

To address the challenges of predeployment sustainability analysis (CH1–CH5),

we developed LEAF^d: a lightweight system that integrates into Terraform workflows to provide carbon projections from .tf files, without requiring source code or running infrastructure.

At its core, LEAF employs a three-layered modeling architecture (Figure 1) that translates IaC definitions into actionable sustainability metrics. This system deconstructs the estimation problem into the following distinct layers:

- The *abstract resource layer* classifies Terraform resources

^dThe complete source code, documentation, and usage examples for the LEAF framework are publicly available on GitHub: <https://github.com/sa4s-serc/LEAF>

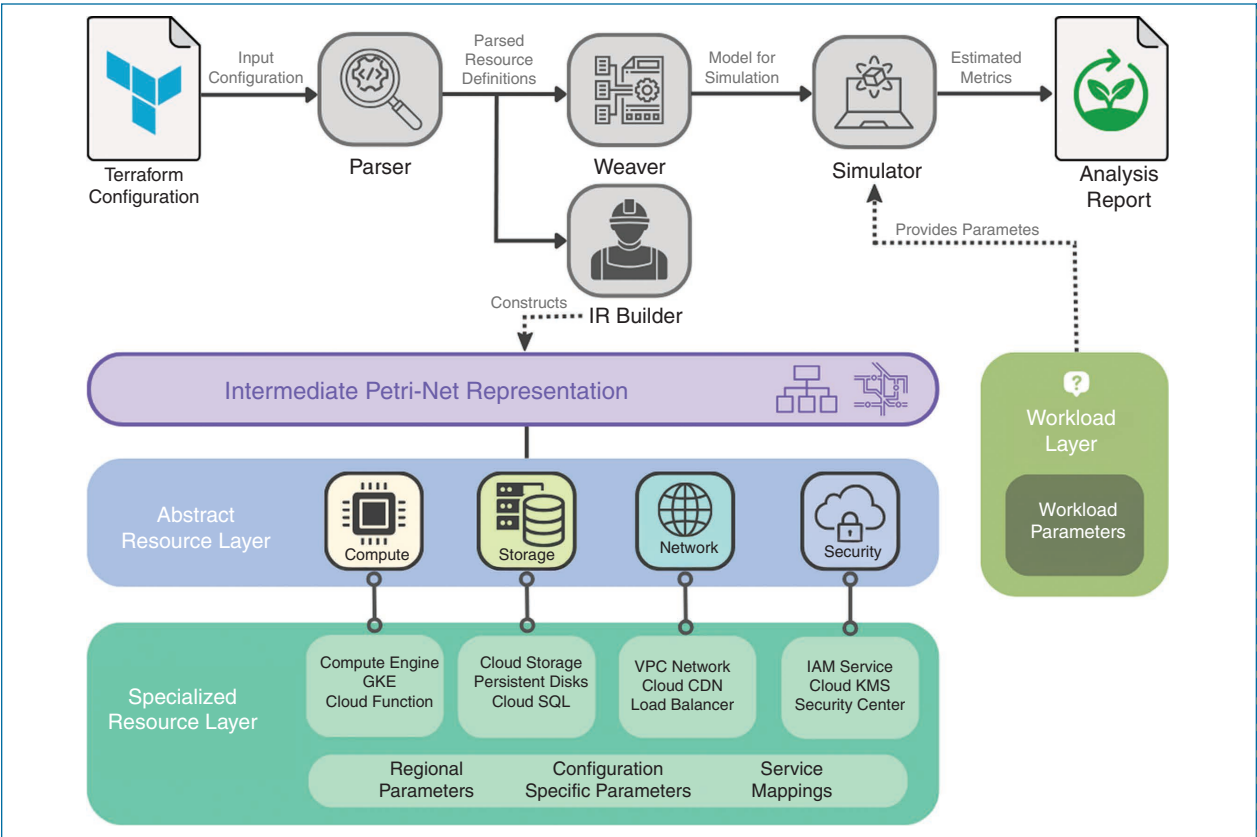


FIGURE 1. LEAF architecture. GKE: Google Kubernetes Engine; CDN: Content Delivery Network; KMS: Key Management Service.

into fundamental categories: compute, storage, network, and security. All resources inherit from a common base resource class that serves two critical functions. First, it provides a robust fallback mechanism: any unknown Terraform resource is mapped to this generic class, ensuring any IaC file can be processed, albeit with coarser precision. Second, it handles non-operational resources (e.g., IAM roles with no direct energy footprint) by mapping them to a near-zero baseline model. This dual-purpose design guarantees that every IaC component is accounted for, ensuring complete coverage without omitting any resource from the final estimate. Each resource is annotated with baseline metrics (e.g., idle power consumption) derived from the hardware power coefficients published in the CCF.⁸ These values are also configurable, enabling practitioners to substitute them with more precise coefficients derived from domain-specific knowledge.

- The *specialized resource layer* maps abstract resources to specific cloud services (e.g., Google Compute Engine), utilizing region-specific carbon intensity factors.¹² It accounts for platform-level behaviors like autoscaling, workload migration, and failover-critical for capturing real-world infrastructure dynamics.
- The *workload layer* introduces usage context by simulating demand based on key parameters like request rate. Based on stakeholder consultations, we identified various production

traffic patterns, including steady-state operational loads, periodic batch processing, and event-driven spikes. For our evaluation, we selected uniform distribution as the baseline pattern to establish reproducible, controlled test conditions while validating LEAF's core estimation accuracy. The workload specification parameters, including specific request per second (RPS) levels can be configured based on historical production data or anticipated operational conditions, allowing teams to model their organization-specific traffic profiles. These inputs drive a time-based simulation of how infrastructure scales and responds to demand. The framework is designed to support future extensions for other workload patterns, such as stochastic models that more closely approximate complex real-world traffic variations.

By integrating these three layers, LEAF constructs a comprehensive model of an IaC deployment. It combines resource definitions, specific service characteristics, and dynamic workload projections to produce estimates.

Processing Pipeline: From IaC to Carbon Metrics

LEAF's analysis pipeline translates Terraform configurations into an actionable model using two core modules: a parser and a weaver.

The parser module ingests Terraform HashiCorp Configuration Language^e files, generating a structured

intermediate representation (IR) that normalizes the infrastructure definition. This provider-agnostic IR captures all resources, attributes, and dependencies, decoupling the parsing logic from the simulation engine for modularity.

Next, the weaver module constructs a layered Petri net model from the IR.¹⁴ This formal construction process is central to our contribution and involves three steps:

- *Resource-to-place mapping:* Each IR resource is mapped to a typed place in the Petri net. This place is annotated with metadata from the specialized resource layer, including its energy profile, capacity, and state.
- *Dependency-to-transition modeling:* Dependencies from the IR (e.g., a compute instance relying on a database) are modeled as transitions and arcs. These define the potential paths a request can take through the system.
- *Modeling downstream load propagation:* The simulation propagates load through the Petri net using a declarative workload specification that encodes both aggregate traffic intensity and per-request behavior. Instead of fixed assumptions, users define request arrival patterns (e.g., steady, bursty, or stochastic) and key interaction ratios, such as the number of database queries, cache hit rates, or API calls per request, that determine how tokens are generated across dependent components. These parameters guide how transactional load fans out downstream, enabling controlled “what-if” analyses without modifying model topology.

^e<https://developer.hashicorp.com/terraform> (last accessed June 15, 2025)

For example, a `google_compute_instance_group_manager` resource becomes a place with dynamic capacity reflecting its autoscaling policy, while its database dependency defines an edge whose load scales with the workload parameters. This translation ensures the Petri Net remains a faithful and reproducible abstraction of the IaC configuration, yet flexible enough to capture realistic workload variations.

The simulation records how tokens flowthrough this network of resources, recording key performance indicators, such as utilization ratio $u_i(t)$, allocated capacity $c_i(t)$, latency $l_i(t)$, and the state of each resource $S_i(t)$. The state reflects the lifecycle of the infrastructure components, such as available, allocated, degraded, etc. This allows LEAF to simulate realistic behaviors, including autoscaling events, failovers, and partial service degradation over time.

This network of resources is modeled as a Petri net, where the connections between components are defined by parameterized arcs. This system of weighted connections forms a directed acyclic graph, enabling LEAF to simulate not just sequential flows but also complex patterns of concurrent demand across the cloud services.

The estimator module is the final stage of the pipeline, responsible for processing the time-series data generated by the simulator. It translates the raw simulation output (e.g., resource utilization traces) into the final energy and carbon projections. This is performed using a three-step calculation for each resource (i) at each time step (t) of the simulation:

1. Calculate energy consumption:

A resource's energy consumption,

$E_i(t)$, is a function of its operational state. This is calculated using a resource-specific energy model, denoted by the function f_e , which takes two main inputs: the request load $R_i(t)$ and the resource's utilization ratio $u_i(t)$. The utilization ratio is the allocated capacity $c_i(t)$ divided by its total provisioned capacity c_i^{total} .

$$u_i(t) = \frac{c_i(t)}{c_i^{\text{total}}}$$

$$E_i(t) = f_e(u_i(t), R_i(t))$$

Rather than a single equation, f_e represents a family of power models derived from the CCF methodology. For example, for a compute resource, the model is conceptually

Energy = (Idle Power + Dynamic Power $\times$ Utilization) $\times$ Time, allowing LEAF to apply tailored power draw models for different resource types.

2. *Convert energy to carbon emissions:* The calculated energy $E_i(t)$ (in kWh) is then converted into carbon emissions, $CO_{2,i}(t)$. This is achieved by multiplying the energy value by the grid carbon intensity factor, γ_i , for the specific cloud region where the resource is deployed.

$$CO_{2,i}(t) = E_i(t) \times \gamma_i$$

The carbon intensity γ_i is measured in grams of CO₂ equivalent per kilowatt-hour (*gCO₂eq/kWh*) and is sourced from public data provided by cloud providers, reflecting the local energy grid's carbon footprint.

3. *Aggregate total impact:* Finally, the module aggregates these

estimates across the deployment graph to produce system-wide projections and presents these key sustainability metrics.

$$E_{\text{total}} = \sum_i \sum_t E_i(t)$$

$$CO_{2,\text{total}} = \sum_i \sum_t CO_{2,i}(t)$$

A detailed description of the modeling architecture, including the Petri net construction, are provided in supplementary material.^f

Putting LEAF to Work

To evaluate LEAF's practical effectiveness in real-world settings, we applied it to three open source cloud projects representing common enterprise architectural patterns. The projects were chosen for their varying complexity, service usage, and infrastructure architecture. Project A^g served as a baseline, using a simple "Hello world" application on Google Cloud Run to test lightweight deployments. Project B^h represented a common platform-as-a-service architecture with a Java Spring Boot application and a cloud SQL database. Project Cⁱ assessed LEAF's ability to model different network and security profiles by using Cloud SQL with a combination of public and private IP configurations.

We tested each project against three simulated workloads to gauge how predictions scaled with demand: low (10 RPS), medium (100 RPS), and high

^fSupplementary material available at <https://tinyurl.com/3z9x9kz3>

^g<https://github.com/mozammal/spring-boot-terraform-cloud-run-demo> (last accessed June 15, 2025)

^h<https://github.com/joaovictorino/terraform-docker-gcp-spring> (last accessed June 15, 2025)

ⁱ<https://github.com/HenryXiloi/demos-gcp/tree/main/cloudsql/private-service-access-PrivateIP-PublicIP/terraform> (last accessed June 15, 2025)

(500 RPS). These levels were chosen to reflect realistic usage ranges seen in production environments after consulting with the stakeholders at Lloyds. For each scenario, we used LEAF to generate predeployment estimates of energy usage and carbon emissions based on the Terraform definitions and input workload parameters.

To validate LEAF's predictions, we compared its outputs against post-deployment measurements from CCF, a widely adopted post-deployment measurement tool.¹⁵ We also include Carbonifer, which to the best of our knowledge is the only existing predeployment IaC-based carbon estimator, to empirically demonstrate the limitations of static analysis approaches that lack workload modeling (CH2). We report Carbonifer only as a static IaC scope/coverage baseline and do not use it for accuracy benchmarking under load; CCF serves as the postdeployment reference. The evaluation focused on three key questions:

1. Does LEAF's simulation approach produce estimates that align with postdeployment measurements?
2. How well does it respond to workload variation?
3. How flexible is it across different types of infrastructure?

Observations

Our evaluation of LEAF across the three projects and varying workloads revealed key insights into its behavior, accuracy, and practical relevance. In project A, emissions estimates increased from 0.54 g CO₂ under low load (10 RPS) to 5.9 g CO₂ at high load (500 RPS). Project B showed emissions ranging from 6 g to 26 g CO₂, while Project C produced the highest emissions, rising from 118.32 g at low load to 402.18 g at high load. These scaling patterns are visualized in Figure 2, and detailed comparisons across tools are presented in Table 1.

In terms of accuracy, LEAF's predictions remained close to post-deployment data from CCF. For

project A, the deviation at high load was 9.26% (5.9 g versus 5.4 g), while at low load it increased to 45.45% (0.54 g versus 0.99 g). Although the absolute difference remained small (0.45 g), this reflects the model's limitation under minimal infrastructure use, where idle activity introduces proportionally larger noise. Carbonifer did not support this project. For project B, LEAF estimated 26 g at high load versus CCF's 22.47 g, a deviation of 15.7%. At low load, LEAF predicted 6 g, while CCF recorded 1.54 g (a 289.61% difference). In contrast, Carbonifer produced a single, load-agnostic value of 3.4 g for the project, underpredicting emissions even at the lowest workload.

For Project C, LEAF showed high accuracy, with a 6.6% deviation at high load (402.18 g versus. 430.7 g) and a 14.29% deviation at medium load (128.58 g versus. 150.7 g). Carbonifer failed to capture workload dynamics, estimating a negligible 0.01 g for all scenarios. Its static, IaC-only model cannot account for

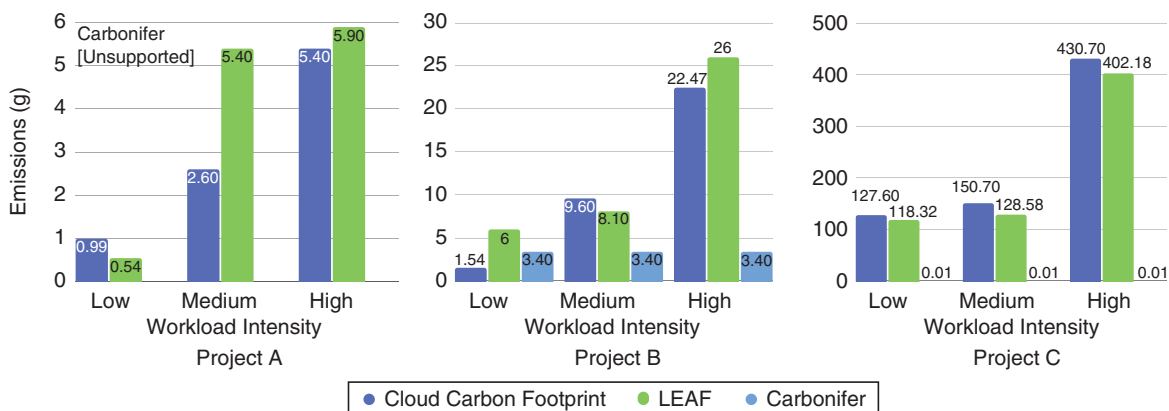


FIGURE 2. Comparison of predeployment carbon estimates from LEAF and Carbonifer against postdeployment measurements from CCF across three applications under low, medium, and high workload intensities.

Table 1. Carbon estimates of different approaches (in g).							
	LEAF			CCF			Carbonifer
	Low	Medium	High	Low	Medium	High	
Project A	0.54	1.3	5.9	0.99	2.6	5.4	[Unsupported]
Project B	6	8.1	26	1.54	9.6	22.47	3.4
Project C	118.32	128.58	402.18	127.6	150.7	430.7	0.01

runtime dynamics like autoscaling, networking, or workload, underscoring the limitations of approaches lacking contextual scaling logic.

In contrast, LEAF accurately captured emissions across diverse workloads and deployment types. Moreover, LEAF proved useful in surfacing “high-impact” IaC-level choices, such as instance type, scaling thresholds, and regional deployments, which had measurable effects on emissions. The ability to analyze these tradeoffs without needing application source code further enhanced its value in enterprise scenarios.

LEAF demonstrates reliable performance under realistic load conditions, showing the highest accuracy in complex deployments, such as Project C (6.6–14.3% deviation). Smaller projects and lighter workloads exhibited greater percentage deviations, reflecting current modeling limits under low utilization. Across all scenarios, LEAF effectively highlighted configuration choices, such as instance types, scaling policies, and region selection that significantly impact carbon emissions. Overall, its reliability improves with increasing workload and system complexity, aligning with its design focus on large-scale deployments. Future work will extend the modeling approach to better capture emissions behavior in smaller workloads.

Threats to Validity

Our model’s accuracy depends on parameters in the abstract and specialized layers, derived from public data and empirical measurements. These may not perfectly reflect instantaneous power drawn across all cloud services. The 24-hour CCF measurement window approximates short-term fluctuations. While base resource fallback ensures complete IaC coverage, its use for primary energy-consuming resources would reduce precision; this had a negligible impact on our evaluation as all such services were mapped to specialized models.

Our study evaluates three projects on Google Cloud. While the framework is designed for extensibility, generalizing the results to other cloud providers (e.g., AWS, Azure) or different application types (e.g., HPC, big data analytics) would require further validation.

Due to the absence of established validation baselines, we employ CCF, though future work could explore alternative validation approaches.

Validation and Key Takeaways

LEAF’s IaC-to-simulation approach enables predeployment, carbon-aware decisions by modeling workload and infrastructure dynamics. Our validation against CCF confirms these

projections are realistic for comparative analysis, addressing challenges CH1–CH5. By modeling how emissions scale with demand and autoscaling, LEAF enables fair pre-deployment comparisons, unlike static IaC analyzers. These relationships between challenges, LEAF’s architectural features, and supporting validation excerpts are summarized in Table 2.

Implications for Research

This research highlights two promising avenues for advancing sustainability modeling in cloud architecture.

- Hybrid carbon estimation frameworks need to be explored that feed live operational data such as actual power draw, scaling events, and resource utilization back into pre-deployment models, creating self-correcting predictors that adapt to real-world deviations.
- Extensible modeling architectures are needed to support emerging cloud services like serverless runtimes, allowing integration of service-specific behavior and carbon profiles to maintain estimator accuracy in a rapidly evolving cloud ecosystem.

Table 2. Challenges and solutions offered by LEAF.

Challenges	LEAF solution	Excerpts from validation
CH1: Reactive versus proactive analysis	LEAF performs predeployment carbon estimation directly from Terraform files, enabling environmental sustainability to be considered during infrastructure planning.	"LEAF's predeployment estimates aligned with postdeployment CCF data within 6.6–14.3% for complex and 15.6–15.7% for moderate workloads, validating its utility for proactive predeployment analysis."
CH2: Insufficient granularity in predeployment estimates	LEAF parses resource-level definitions and maps them to energy profiles of various types of cloud resources, allowing fine-grained estimation across compute, storage, network, and security components.	"LEAF surfaced high-impact provisioning choices early, such as instance type and autoscaler settings, highlighting their influence on projected emissions."
CH3: Dependency on code or agents	LEAF operates entirely on Terraform configuration files and does not require access to application source code or deployment agents.	"The ability to simulate infrastructure behavior without requiring application source code made LEAF especially useful in enterprise settings."
CH4: The need for incremental guidance in sustainable IaC development	LEAF supports simulation of different workloads and configurations, enabling comparison of carbon impact under multiple IaC variants.	Across all three projects, LEAF modeled workload scaling accurately—for instance, emissions in the Project A increased from 0.54 g CO ₂ at 10 RPS to 5.9 g CO ₂ at 500 RPS, while the project C rose from 118.32 g to 402.18 g—demonstrating sensitivity to workload-driven variations.
CH5: Disconnect between development practices and sustainability data	LEAF integrates sustainability analysis directly into Terraform workflows, using region-specific carbon intensity via cloud APIs.	"LEAF gave our teams the ability to assess region-specific tradeoffs across latency, cost, and emissions directly within our existing Terraform workflows, without requiring changes to our infrastructure provisioning process." — Cloud Engineering Team, Lloyds

Implications for Practice

The analysis of our results shows that LEAF enables cloud architects and DevOps teams to systematically incorporate sustainability considerations into their predeployment workflows. Its integration into infrastructure planning offers several actionable benefits:

- *Embedding of carbon estimates into the planning workflow:* Carbon estimates are generated using LEAF alongside cost and performance metrics during the Terraform planning stage, ensuring that high-emission configurations are identified and addressed before any resources are provisioned.
- *Leveraging LEAF to run "what-if" scenarios:* Emission impacts are compared using LEAF across

various virtual machine sizes, autoscaling rules, and geographic regions, enabling the most sustainable option to be selected without compromising functionality.

- *Incorporation of real-world usage patterns:* Real-world workload peaks, such as seasonal traffic surges or end-of-month reporting periods, are modeled within LEAF to validate that scaling settings and capacity plans minimize emissions during high-load periods, thereby embedding sustainability into predeployment decision making.

Together, these capabilities shift carbon analysis from a reactive, postdeployment task to a proactive element of deployment decisions.

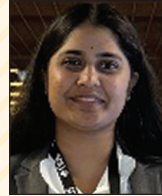
In summary, LEAF enables predeployment, carbon-aware decisions by turning Terraform into workload-aware simulations integrated into standard workflows. External validation against post-deployment measurements confirms the projections align well with observed emissions, turning sustainability into a first-class provisioning criterion. Moving forward, we plan to integrate runtime feedback loops to refine our predictions continuously, broaden our service coverage, and embed LEAF into common developer tool chains, ensuring that every infrastructure decision is both performance-aware and environmentally friendly. 🌱

References

1. "Energy and AI," Int. Energy Agency, Paris, France, 2025. [Online]. Available: <https://www.iea.org/reports/energy-and-ai>



PRAKHAR SINGHAL is a research assistant in the Software Engineering Research Center, International Institute of Information Technology Hyderabad, Hyderabad 50032, India. His research interests include self-adaptive systems and sustainable cloud infrastructure. Singhal received his B.Tech and MSc by research in computer science and engineering from the International Institute of Information Technology Hyderabad. Contact him at prakhar.singhal@research.iiit.ac.in.



AKHILA MATATHAMMAL is a Ph.D. scholar at the International Institute of Information Technology Hyderabad, Hyderabad 50032, India. Her research interests include sustainable AI systems and software architecture, optimizing energy, cost, and performance at both design and runtime. Matathammal received her B.Tech at Sridevi Women's Affiliated to JNTUH. Contact her at akhila.matathammal@research.iiit.ac.in.



SHAUNAK BISWAS is an undergraduate researcher at the Software Engineering Research Center, International Institute of Information Technology Hyderabad, Hyderabad 50032, India. His research interests include green software engineering, with focuses on creating self-adaptive architectures to embed sustainability into Machine Learning Operations pipelines and developing frameworks to estimate the carbon footprint of cloud deployments. Biswas received his B.Tech and MSc by research in computer science and engineering from the International Institute of Information Technology Hyderabad. Contact him at shaunak.biswas@research.iiit.ac.in.



CH PAVAN HARSHIT is a research engineer at the Software Engineering Research Center, International Institute of Information Technology Hyderabad, Hyderabad 50032, India. His research interests include software systems, data engineering, and applied machine learning. Harshit received his B.E. in computer science engineering from Chaitanya Bharathi Institute of Technology. Contact him at pavan.harshit@research.iiit.ac.in.



KARTHIK VAIDHYANATHAN is an assistant professor at the Software Engineering Research Center, International Institute of Information Technology Hyderabad, Hyderabad 50032, India. His research interests include the intersection of software engineering and AI with specific focus on engineering self-adaptive and sustainable software systems. Vaidhyanathan received his Ph.D. in computer science from Gran Sasso Science Institute, Italy. He is an editorial board member of *IEEE Software*. Contact him at karthik.vaidhyanathan@iiit.ac.in.



RAVI KAPPAGANTU is the head of Architecture and Innovations at Lloyds Technology Centre Hyderabad, Hyderabad 50032, India. His research interests include tech sustainability, quantum computing, AI, and deep learning. Kappagantu received his Master of Computer Applications (M.C.A.) from the University of Hyderabad. Contact him at ravi-kumar.kappagantu@lloydsbanking.com.





SRINIVAS KATAKAM is the senior director of Enterprise Applications at Lloyds Technology Centre Hyderabad, Hyderabad 50032, India. His research interests include green computing, quantum computing, and AI/machine learning programs. Katakam received his B.E. from the National Institute of Technology, Tiruchirappalli. Contact him at srinivas-v.katakam@lloydsbanking.com.



JITENDRA CHOUGALA is the head of Engineering at Lloyds Technology Centre Hyderabad, Hyderabad 50032, India. His research interests include cloud computing, large-scale distributed systems, and sustainable software engineering. Chougala received his B.E. in mechanical engineering from Karnatak University. Contact him at jitendra.chougala@lloydsbanking.com.

2. P. Lago, S. A. Koçak, I. Crnkovic, and B. Penzenstadler, "Framing sustainability as a property of software quality," *Commun. ACM*, vol. 58, no. 10, pp. 70–78, Sep. 2015, doi: [10.1145/2714560](https://doi.org/10.1145/2714560).
3. F. Qiao, A. Mohammadi, J. Cito, and M. Santolucito, "Statically inferring usage bounds for infrastructure as code," in *Proc. 16th Int. Conf. Verified Softw. Theories, Tools Experiments*, 2025, pp. 84–95.
4. "Carbon aware SDK - Green Software Foundation." Green Software Foundation. Accessed: Dec. 11, 2025. [Online]. Available: <https://carbon-aware-sdk.greensoftware.foundation/>
5. L. Lannelongue, J. Grealey, and M. Inouye, "Green algorithms: Quantifying the carbon footprint of computation," *Adv. Sci.*, vol. 8, no. 12, 2021, Art. no. 2100707, doi: [10.1002/advsc.202100707](https://doi.org/10.1002/advsc.202100707).
6. S. Kosbar and M. Hamdaqa, "Smells-sus: Sustainability smells in IaC," in *Proc. IEEE/ACM 22nd Int. Conf. Mining Softw. Repositories (MSR)*, Ottawa, ON, Canada, 2025, pp. 801–812, doi: [10.1109/MSR66628.2025.00117](https://doi.org/10.1109/MSR66628.2025.00117).
7. "Sustainability." Lloyds Banking Group. Accessed: Jun. 12, 2025. [Online]. Available: <https://www.lloydsbankinggroup.com/sustainability.html>
8. "Carbon footprint." Google Cloud. Accessed: Dec. 11, 2025. [Online]. Available: <https://cloud.google.com/carbon-footprint?hl=en>
9. "Customer carbon footprint tool." Amazon Web Services. Accessed: Dec. 11, 2025. [Online]. Available: <https://aws.amazon.com/sustainability/tools/aws-customer-carbon-footprint-tool/>
10. V. Cortellessa, D. Di Pompeo, and M. Tucci, "Exploring sustainable alternatives for the deployment of microservices architectures in the cloud," in *Proc. IEEE 21st Int. Conf. Softw. Archit. (ICSA)*, 2024, pp. 34–45, doi: [10.1109/ICSA59870.2024.00012](https://doi.org/10.1109/ICSA59870.2024.00012).
11. "Software Carbon Intensity (SCI) specification." Green Software Foundation. Accessed: Dec. 11, 2025. [Online]. Available: <https://sci.green-software.foundation/>
12. "Carbon free energy for Google Cloud regions." Google Cloud. Accessed: Dec. 11, 2025. [Online]. Available: <https://cloud.google.com/sustainability/region-carbon>
13. "GitHub - Green-Software-Foundation/real-time-cloud: Real time energy and carbon standards for cloud providers." GitHub. Accessed: Dec. 11, 2025. [Online]. Available: <https://github.com/Green-Software-Foundation/real-time-cloud>
14. J. L. Peterson, "Petri nets," *ACM Comput. Surv.*, vol. 9, no. 3, pp. 223–252, 1977, doi: [10.1145/356698.356702](https://doi.org/10.1145/356698.356702).
15. "Bringing green cloud optimization to a green energy business." Thoughtworks. Accessed: Dec. 11, 2025. [Online]. Available: <https://www.thoughtworks.com/clients/Bringing-green-cloud-optimization-to-a-green-energy-business>
16. Z. Li et al., "A survey on modeling energy consumption of cloud applications: Deconstruction, state of the art, and trade-off debates," *IEEE Trans. Sustain. Comput.*, vol. 2, no. 3, pp. 255–274, Jul./Sep. 2017, doi: [10.1109/TSUSC.2017.2722822](https://doi.org/10.1109/TSUSC.2017.2722822).